Privacy-Preserving KYC/AML

Project plan and implementation guide
CS670: Cryptographic Techniques for Privacy Preservation

Version	Planning draft 1 (nothing implemented yet)
Scope	Problem statement, architecture, per-module protocol and implementation details, attack lab, testing, milestones, optional extensions
Base	Course modules: MPC, Secure Memory Access (ORAM/DORAM), Zero-Knowledge Proofs
Reading guide	Orange boxes mark optional or stretch work. Everything outside those boxes is the core plan.
Citations	Papers listed in Section 14 were named from memory and must be verified (titles, authors, years) before they go in the report.

1. Problem statement and use case

Cross-institution KYC/AML compliance without sharing customer data.

Banks must (a) screen customers against sanctions and watchlists, (b) verify identity claims such as age or residency, and (c) detect money-laundering patterns, such as structuring, that only appear when several institutions' data is combined. Today this forces a bad choice: either everyone ships raw personal data to a central party (a breach honeypot that also sees everything), or institutions cannot cooperate and cross-bank patterns go unnoticed. Regulation demands both compliance and data protection.

Goal. Build a working prototype showing that secure multiparty computation, oblivious memory access and zero-knowledge proofs deliver the compliance outcomes while raw data never leaves its owner. Build a deliberately flawed naive system (v0) next to the private system (v1), run the same attacks on both, and explain which course property blocks each attack.

Why this use case fits the course

- It has natural two-party, three-party and many-prover/one-verifier settings, so every course topic has a real job.
- Each course module maps to one concrete KYC task (Section 3), so the project is organised around the syllabus rather than around a single paper.
- KYC has real, well-known failure modes (breaches, hash dictionary attacks, access-pattern leakage, replay), which makes the attack lab credible.

2. What the course covers (the base we build on)

Module	Topics from the slides
MPC	Expressibility, minimum parties, threat model, maximum adversaries, performance (local computation, rounds, bandwidth); additive, replicated (PRG zero-sharing) and Shamir sharing; Lagrange coefficients and degree reduction; Du-Atallah/Beaver multiplication with preprocessing; 1-out-of-2 OT, malicious fix, random OT, IKNP extension; GMW; Blum coin toss; DH-based PSI and hash-to-group; garbled circuits (point-and-permute, GRR3, free-XOR); GC vs secret sharing; Yao/Boolean/additive share conversions (Y2B, B2Y, B2A); OT-based Du-Atallah triples; simulator-based security proofs
Secure memory access	Why encryption alone leaks access patterns (cloud storage, genomic data, binary search); ORAM definition; linear-scan ORAM; square-root ORAM; hierarchical ORAM with oblivious sort reshuffle; tree ORAM with evictions and recursive position map; memory access on secret shares; oblivious read/write with one-hot vectors; read-only and write-only memory; DORAM from single-function memory with 3-party dot products; DPFs as the missing piece
ZKP	Graph 3-colouring example; commitments (hiding, binding); sigma protocols; group theory refresher (subgroup membership checks); Schnorr protocol; completeness, special soundness, special honest-verifier ZK; proof of knowledge and extractors; Fiat-Shamir; MPC-in-the-head with 2-of-3 view opening and 1-private protocols

Not in the syllabus, so excluded from the core: BBS+ and Camenisch-Lysyanskaya signatures, pairings, accumulators, zk-SNARKs, Bulletproofs. Where one of these would be the production answer, the report mentions it as future work only.

3. System overview: four modules and two systems

Module	KYC task	Course tools	Parties
A. Sanctions screening	Bank checks a customer against the regulator's watchlist. List stays hidden; the bank learns only whether its own customers match.	DH-based PSI, hash-to-group, group membership checks	Bank (receiver), Regulator (sender)
B. Cross-bank AML analytics	Is a customer's combined deposits across banks above a threshold, and what is their risk score, without revealing per-bank amounts or the model.	Additive shares, Du-Atallah/Beaver triples, OT, garbled circuits, share conversion	Banks, servers S0 and S1, Regulator
C. Private registry and document access	C1: banks keep encrypted KYC documents on an untrusted cloud. C2: query a shared KYC-status registry without revealing which record.	ORAM (linear scan, square-root), 2-server read-only DORAM via one-hot dot products	Bank/client, Cloud, servers S0 and S1
D. Verifiable claims	Customer proves 'age ≥ 18', 'nationality in allowed set', 'KYC level ≥ 2' about issuer-signed committed attributes without showing documents.	Pedersen commitments, Schnorr, DLEQ, OR-proofs, Fiat-Shamir, MPC-in-the-head	Customer, Issuer, Bank (RP)

OT is the shared building block of A (optional OT-based PSI), B and C, so it is implemented once in the core library.

The two systems

- **v0 (naive, deliberately flawed):** central SQLite database of full PII with SHA-256(national ID) as the join key; watchlist check is `sha256(id) in set`; risk engine receives raw per-bank amounts; registry queried by plaintext index; bearer-token sessions; document upload as 'proof' of age.
- **v1 (private):** Modules A to D.
- **Attack lab:** each attack runs against v0 (succeeds) and v1 (fails or is reduced); the report names the course property responsible.

Threat model

- The core is **semi-honest**, matching how most of the course protocols are analysed. Security is argued with simulators (slides 69-70).
- Malicious-adversary behaviour is shown only in the attack lab (malicious OT receiver, cheating ZK prover) and is not claimed for the whole system.
- Out of scope: network and side-channel attacks, real identity proofing at issuance, fuzzy name matching, UI trust, key management.

4. Software architecture

Parties are Python objects. Every message passes through a `Channel` that records the bytes, so each party's **view** (slide 30: the messages sent and received) is available. The views drive the transcript viewer in the demo, the simulator tests, and leakage checks, since a semi-honest adversary is simply a party reading its own view. Optionally wrap channels in localhost TCP to measure real bandwidth.

```
crypto/        group.py  fiat_shamir.py  commit.py  sharing.py  ot.py  gc.py  dpf.py (optional)
zkp/           schnorr.py  dleq.py  orproof.py  range.py  credential.py  mpcith.py
psi/           dh_psi.py  ot_psi.py (optional)
mpc/           beaver.py  dotprod.py  gc_compare.py  pipeline.py
oram/          linear.py  sqrt.py  tree.py (optional)  doram_ro.py
kyc_naive/     v0 services (SQLite, plaintext checks)
kyc_private/   v1 services (issuer, wallet, bank, regulator, servers)
attacks/       harness.py + one file per attack, each runnable against v0 and v1
tests/         known-answer, soundness, simulator and leakage tests
bench/         proof size and time, GC vs shares, ORAM bandwidth
demo/          side-by-side CLI (optional: small web dashboard)
docs/          threat model, protocol specs, paper-to-module table, report
```

Technology choices

- **Language:** Python 3. Arbitrary-precision integers and `pow(a, e, p)` make a mod-p Schnorr group easy; `gmpy2` optionally speeds up modular exponentiation.
- **Hand-written:** group helpers, Fiat-Shamir, commitments, sigma protocols, OT, garbled circuits, secret sharing, PSI, ORAM. **Library allowed:** hashing (SHA-256), AES or ChaCha as a PRG, randomness (`secrets`), and test tooling. Confirm with the instructor that this split is acceptable.
- **Testing:** pytest plus hypothesis for property-based tests.

5. Shared cryptographic core

5.1 Group

Use a Schnorr group: prime p, prime q dividing p-1, and a generator g of order q (the slides use multiplication modulo a prime). Use a standard 2048/256-bit group (for example RFC 5114) or generate one. The group class provides:

- `exp, mul, inv`.
- `is_member(h)`: require 1 < h < p and `pow(h, q, p) == 1`. **Every verifier calls this on every received group element** (slides 13-15: how does the verifier check that h has order dividing q?). Skipping it is attack 13.
- `hash_to_group(x)`: expand with SHA-256 to the length of p, reduce mod p, raise to the cofactor (p-1)/q, reject 1.
- Pedersen's second generator h = hash_to_group('pedersen-h'), so no one knows $\log_g h$.

Performance note: a 256-bit-exponent modular exponentiation in a 2048-bit group is about 1 ms; hash-to-group with cofactor clearing is about 10 ms. Keep demo sets to a few thousand elements. Make the group a pluggable backend if you want a faster curve later.

5.2 Fiat-Shamir transcript

`Transcript(domain)` accepts length-prefixed, domain-separated appends and outputs a challenge mod q. The **strong** version appends the group parameters, the statement, the verifier context (RP id, nonce, timestamp) and all first messages. A deliberately **weak** version (omits context, or truncates the challenge to t bits) exists only for the attack lab.

5.3 Commitments

Pedersen commitment $C = g^m h^r$. It is perfectly hiding and computationally binding; the slides define computational hiding and binding, so the report notes that this is a stronger hiding variant. A hash commitment H(m || r) is also provided for MPC-in-the-head views.

5.4 Secret sharing

- Additive over $Z_{2^{32}}$ (for MPC arithmetic) and over Z_q (for the ZKP side).
- **Optional:** replicated 2-out-of-3 shares with PRG-derived zero sharing (slides 19-23); required for MPC-in-the-head (D3).

5.5 Base oblivious transfer

1-out-of-2 OT in the Naor-Pinkas style. This is the "tie Bob's hands" fix from slide 33:

```
Sender:   pick c, send C = g^c
Receiver: choice bit b; pick k; PK_b = g^k; PK_{1-b} = C / PK_b; send PK_0
Sender:   PK_1 = C / PK_0; for i in {0,1}: pick r_i,
          send ( g^{r_i} , H(PK_i^{r_i}) XOR m_i )
Receiver: m_b = c_b XOR H( (g^{r_b})^k )
```

The receiver cannot know the discrete logs of both keys without knowing log C, which is exactly why the naive protocol is breakable by a malicious receiver (attack 6). Also provide random OT, and batch OT calls for GC input labels and Beaver triple generation.

Optional: OT extension (IKNP, slides 38-44)
• kappa = 128 base OTs with the roles swapped, then extend to millions of OTs using a PRG and a correlation-robust hash.
• Demonstrate the slide's warning that reusing the string s leaks correlations, and discuss that IKNP is not secure against malicious parties without extra checks.
• Benefit: Module B triple generation and GC input OTs drop from hundreds of base OTs to a constant number.

5.6 Garbled circuits

- 128-bit wire labels. Garbler sends its own input labels directly; evaluator's input labels are delivered by OT.
- Optimisations in the order the slides introduce them: **point-and-permute** (colour bit = LSB of label), **free-XOR** (global offset Δ with LSB(Δ) = 1), **GRR3** (AND gates send 3 ciphertexts).
- Gate encryption: `H(La || Lb || gate_id)` as a one-time pad, via truncated SHA-256.
- Circuits are built in Python: ripple-carry adder mod 2^{32} and a comparator. **Every garbled evaluation is tested against plaintext evaluation on random inputs.**

6. Module D: verifiable KYC claims (ZKP)

6.1 Sigma-protocol toolkit

Protocol	Statement	Implementation notes
Schnorr	I know w with $h = g^w$	$a = g^r$; challenge e; $z = r + e \cdot w \bmod q$; verify $g^z = a \cdot h^e$
Pedersen opening	I know (m, r) with $C = g^m h^r$	Two-witness variant: z_m, z_r
DLEQ (Chaum-Pedersen)	Same w in g^w and u^w	Used for the scoped nullifier
OR-proof (CDS)	Statement A or statement B	Simulate the false branch, split the challenge $e = e_A + e_B$
Range proof	Committed v lies in $[0, 2^k)$	Bit commitments, one OR-proof per bit, product check (below)

Every protocol exists in **interactive** and **Fiat-Shamir** form, and each ships with its **simulator** (pick z and e, then set $a = g^z \cdot h^{-e}$) and its **extractor** ($w = (z - z')/(e - e')$ from two accepting transcripts with the same a). This lets you demonstrate completeness, special soundness and special honest-verifier ZK exactly as defined on slides 18-22.

Range proof detail

To prove age ≥ 18 from C_{dob}, prove cutoff - dob ∈ $[0, 2^{16})$:

- Commit to each bit: $C_i = g^{b_i} h^{r_i}$.
- OR-proof per bit that C_i opens to 0 or 1 (C_i opens to 0, or C_i/g opens to 0).
- Verifier checks $\Pi C_i^{2^i}$ equals $g^{cutoff} \cdot C_{dob}^{-1} \cdot h^{\delta}$, where the prover supplies δ to balance the randomness.
- Proofs are tens of KB. This is the natural place to note why Bulletproofs exist (future work).

6.2 Credential and presentation (built only from sigma + Fiat-Shamir)

Issuance

After offline verification, the wallet picks a secret key *sk* and commits to every attribute including *sk* itself: C_{sk}, C_{dob}, C_{nat}, C_{level}. The Issuer checks the openings and produces a plain **Schnorr signature** on H(commitments || expiry || issuer_id). That signature is the sigma protocol plus Fiat-Shamir applied to signing.

Presentation

The RP sends (rp_id, nonce, timestamp, predicate list). The wallet returns the credential and a single combined proof under one challenge:

```
e = H( params || ctx || commitments || signature || all first-messages )
```

- age ≥ 18 (range proof) and KYC level ≥ 2 (range proof).

- nationality in the allowed set (OR over the set).
- **Scoped nullifier:** $\nu = \text{H2G}(\text{rp_id})^{sk}$, with a DLEQ-style proof that the sk inside C_{sk} is the exponent of ν. Verification equations: $g^{z_{sk}} h^{z_s} = a_1 \cdot C_{sk}^{\ e}$ and $u^{z_{sk}} = a_2 \cdot \nu^e$.
- The RP stores ν and rejects duplicates, which gives **sybil resistance**. Different RPs see unrelated ν values under the DDH assumption.
- The proof is bound to (rp_id, nonce, timestamp), which defeats replay (attack 9). Tying sk to C_{sk} gives holder binding (attack 10).

Honest limitations (put these in the report)

- A Schnorr-signed commitment is a static value, so two RPs could link a *reused* credential. Mitigation: the Issuer hands out a batch of one-time credentials with fresh commitments but the same sk. The nullifier stays consistent per RP and RPs cannot link each other.
- The Issuer can still link all issuance and usage because issuance is not blind. Blind issuance and re-randomisable signatures (BBS+, CL) are out of syllabus and listed as future work.

6.3 MPC-in-the-head age proof (the course-heavy comparison)

Prove the same age predicate a second way, using the slides' MPC-in-the-head construction:

- A 3-party **replicated-sharing** protocol with PRG-derived randomness (the machinery of slides 19-23, reused).
- A comparator circuit over the secret date of birth: XOR is local; AND uses replicated multiplication plus a zero-sharing correction.
- The prover runs all three parties in its head and commits to each party's view and output shares. Fiat-Shamir selects which two views to open. The verifier recomputes those two parties' messages and checks consistency and output = 1.
- Opening two of three views is the "two random transcripts" step on slide 31. Soundness error is 2/3 per round, so about 69 rounds give 2^{-40} and about 137 give 2^{-80}.
- **Where the properties come from** (slide 38): zero-knowledge from the privacy of the MPC protocol against two corrupted parties; soundness from the consistency checks across views.

Then benchmark proof size and prove/verify time for the sigma range proof versus the MPCitH proof of the same claim.

Optional extensions for Module D
• Use the slides' 1-private two-party variant (open one party's view and one channel) as a simpler first MPCitH implementation, then upgrade to 3 parties.
• Frozen Heart reproduction: only after reading the paper, replicate the weak Fiat-Shamir break on a compound (OR / multi-statement) proof where the statement is adaptively chosen.
• Batch one-time credentials with an expiry schedule, and a simple issuer-side revocation list as a v0-style baseline to discuss why privacy-preserving revocation is hard.
• Blind issuance and BBS+ as a documented future-work design only.

7. Module A: sanctions screening (PSI)

DH-based PSI with the Regulator as sender (watchlist W) and the Bank as receiver (customer identifiers Q). Each party holds a secret exponent:

```
1. Bank -> Regulator:  { H(q)^a : q in Q }                      (order preserved)
2. Regulator -> Bank:  { H(q)^(ab) }  (same order)  and  shuffled { H(w)^b : w in W }
3. Bank computes (H(w)^b)^a for each w, intersects locally.
   Bank learns: which of ITS customers matched, and |W|.   Regulator learns: |Q|.
```

- **H hashes into the group** (slides 47-48). The report explains the ratio leak if hashing is done with a plain integer hash (attack 3).
- Canonicalise identifiers (lowercase, strip, e.g. 'name|dob|nationality') and state that fuzzy matching is out of scope.
- Verify group membership of every received element (attack 13). Cache H(w) on the Regulator side.

- **Enumeration attack (attack 12), stated honestly:** a malicious bank can submit the whole identifier space as Q and learn W. PSI cannot prevent this cryptographically. v1 mitigates it with authenticated queries, a cap on |Q| per time window and an audit log, but does not eliminate it.

Optional: OT-based PSI
• Implement the OT-based PSI sketched on the slides (4-out-of-1 OT style) as a second PSI and compare it with DH-PSI on computation versus communication.
• Circuit-PSI (output only secret shares of the match bit) to feed Module B directly; otherwise the demo assumes the aligned customer set comes from Module A's output.

8. Module B: cross-bank AML analytics (hybrid MPC)

Pipeline: secret sharing for linear work, garbled circuits for comparison, which is the "use both" argument of slides 58-64.

Step	What happens	Course slide topic
1. Share	Each bank splits its features (deposit amounts, counts) additively between S0 and S1. Sums over banks are local and free.	Additive shares; addition needs no communication
2. Score	Regulator's model weights w are private, features x are shared. <w, x> is computed with Du-Atallah/Beaver. Online: parties open d = x - a and e = w - b, then z = c + d·b + e·a + d·e. Openings are uniformly random because a and b are random masks (the security argument).	Beaver / Du-Atallah triples; preprocessing vs online
3. Triples	Offline: OT-based multiplication. For each bit i of b the sender offers (r_i, $r_i + a \cdot 2^i$); the receiver chooses by b_i; receiver sums what it received, sender's share is $-\Sigma r_i$.	OT to produce Du-Atallah triples (slide 72)
4. Compare	Convert additive shares into GC inputs; the circuit adds them mod 2^{32} and compares with threshold T; output is only the flag. S0 garbles, S1 evaluates (OT for S1's bits).	GC for non-linear gates; share conversion

- Report offline versus online cost separately, plus OT count, ciphertext count and rounds.
- Use the slides' properties template for each protocol: expressibility, minimum parties, threat model, maximum adversarial parties, performance in (g, m, d).
- A pure-secret-sharing comparison (bit-decomposition with Boolean shares) versus the GC comparison gives the GC-versus-secret-sharing benchmark.

Optional extensions for Module B
• 3-party replicated-sharing version (honest majority, no OT in the online phase), compared with the 2-party Du-Atallah version.
• GMW-style evaluation of the comparator over Boolean shares with 4-out-of-1 OT for AND gates, as a third approach to compare.
• Explicit share conversions (Y2B free via colour bit, B2Y by OT, B2A) in place of ad hoc conversion code.
• Shamir-share version with Lagrange degree reduction, for a threshold-of-banks deployment.

9. Module C: private registry and document access (ORAM/DORAM)

9.1 C1: outsourced encrypted KYC documents (client-server ORAM)

- **Baseline:** linear-scan ORAM (read and rewrite everything every access).
- **Square-root ORAM:** random permutation known to the client, a stash scanned on every access, fake (dummy) items on stash hits, reshuffle every √n accesses (slides 28-35).
- The server logs physical accesses. **Test (slide 15):** logical sequences [1,1,1,1,1,1] and [1,2,3,4,5,6] must give indistinguishable physical traces; run many trials and apply a chi-squared test.
- Motivating attack (5): frequency analysis of physical accesses on a Zipf workload, showing an adversary beating random guessing against plain encrypted storage.

Optional: hierarchical and tree ORAM
• Tree ORAM with buckets, a path invariant, a position map, two-random-buckets-per-level evictions and a recursive position map (slides 45-55).
• Hierarchical ORAM with oblivious-sort reshuffle (slides 36-44) as a theory-and-small-demo item rather than a full implementation.
• Bandwidth-versus-n comparison of linear, square-root and tree ORAM.

9.2 C2: shared KYC-status registry (2-server read-only DORAM)

Slides 59-62: for read-only memory, both servers can hold the plaintext database, and a read becomes a dot product with shares of a one-hot vector.

```
Client knows index i.
  pick random v0, v1 with v0 + v1 = e_i (one-hot vector)
  send v0 to S0, v1 to S1
Each server computes <D, v_j> locally (no MPC needed).
Client adds the two replies and gets D[i].  Neither server learns i.
```

- Cost is O(n) communication per read; report it. Registry updates come from the Regulator, who knows the index, so they are plain epoch updates.
- Contrast with the slide 73 observation that a naive (2+1)-dot-product DORAM read costs O(n) per access, and explain why the 3-party invariant-resetting design avoids this.

Optional: DPF and full DORAM
• Replace the one-hot vectors with a **distributed point function** (GGM tree plus correction words) so the query size drops from O(n) to O(log n). The slides call DPFs 'the missing piece'.
• Read/write DORAM from single-function memories with the 3-party dot product and the invariant-resetting update (slides 65-79).
• Oblivious write via shares of a standard-basis vector (slide 57).

10. v0: the naive system

Real, deliberately flawed services so that the attacks genuinely work:

- SQLite table of full PII; join key is SHA-256(national ID).
- Watchlist check is `sha256(id) in set`; the watchlist is shipped to banks (or customer IDs to the regulator).
- Central risk engine receives raw per-bank amounts.
- Registry and document store queried by plaintext index with server-side access logging.
- Bearer-token sessions; age 'proof' is a document upload; no nonce or RP binding.

11. Attack lab

Harness: `Attack.run(target) -> {success, evidence, metric}`. The runner executes each attack on v0 and v1 and prints a side-by-side table; every v1 failure cites the property that stopped it.

#	Attack	Mechanism	Success metric	v1 defence / course concept
1	PII breach	Dump v0 database; attempt the same on all v1 server states	Fields recovered	No party holds full data; MPC/ZK security
2	Hash dictionary	Brute-force SHA-256(id) over a reduced ID space; extrapolate from measured hashes/s	IDs recovered	DH-PSI with secret exponents
3	Non-group hash ratio leak	PSI variant that hashes to integers, not into a group	Relation learned	Hash-to-group (slides 47-48)
4	Amount snooping	Server reads every bank's raw inputs	Amounts seen	Additive shares + GC comparison
5	Access-pattern inference	Frequency analysis on a Zipf workload	Attacker accuracy vs chance	ORAM / DORAM

#	Attack	Mechanism	Success metric	v1 defence / course concept
6	Malicious OT receiver	Picks both keys as public/private pairs, learns both messages	Both m0 and m1 recovered	Naor-Pinkas-style constraint (slide 33)
7	Nonce reuse	Same r with two challenges, then run the extractor	Secret key recovered	Deterministic nonces; special-soundness extractor as attack
8	Short-challenge forgery	t-bit challenge: guess e, simulate transcript, accept	Forged proof accepted in about 2^t tries	Full-length challenge (slides 17-20)
9	Replay	Resubmit a captured proof to another RP or session	Proof accepted	Nonce, RP id in the FS hash
10	Credential lending	Bob presents Alice's credential	Check passes?	sk bound inside C_{sk}, proved in ZK
11	Sybil accounts	One user opens N accounts at one RP	Accounts created	Scoped nullifier $\nu = H2G(rp)^{sk}$
12	PSI enumeration	Submit an inflated query set to learn the watchlist	Entries learned	Partially mitigated: query cap, auth, audit log. Reported as a residual limit
13	Invalid-group input	Send an element outside the order-q subgroup	Key bits leaked / proof accepted	is_member checks (slides 13-15)

Attacks 7, 8 and 9 make the best live demos; attack 7 turns the special-soundness extractor on the slides into a literal key-recovery break. Correction to an earlier draft: omitting the statement from the Fiat-Shamir hash is not by itself a forgery for a single fixed Schnorr statement; the real break (Frozen Heart) needs adaptively chosen statements in compound proofs, which is why the plain 'weak FS' attack here is the short-challenge forgery.

Optional attacks
• Frozen Heart-style forgery on a compound proof (see Module D optional box).
• Lattice (hidden number problem) attack on Schnorr signatures with biased nonces, to complement attack 7; cite Howgrave-Graham and Smart.
• IKNP correlation leak when s is reused (slides 43-44).
• Malicious garbler sending a wrong circuit, showing why GC needs extra checks outside the semi-honest model.

12. Testing and evaluation

Property	How it is tested
Completeness	Honest runs always accept (property-based tests over random witnesses)
Soundness	Flipping any byte of any proof field must reject; wrong witness fails; extractor recovers w from two accepting transcripts
Zero-knowledge	Simulator transcripts verify; Kolmogorov-Smirnov test on z over many runs shows real and simulated transcripts are indistinguishable
Correctness	GC output equals plaintext circuit output; OT output equals chosen message; Beaver product equals plaintext product; PSI output equals plain set intersection
Leakage	Chi-squared test on ORAM physical traces; check that PSI and MPC views contain nothing beyond the claimed leakage (|Q|, |W|, the flag)
Performance	Modexp counts and time; proof size and prove/verify time (sigma vs MPCitH); GC vs secret-sharing comparison; offline vs online cost; ORAM bandwidth vs n

- **Written security arguments:** one simulator-based sketch per module, following slides 69-70 (show a PPT simulator that produces each party's view from its input and output only).
- **Limitations section:** prototype, not production; semi-honest core; linkable credential reuse; non-blind issuance; PSI enumeration; out-of-scope list.

13. Build order, cut line and optional work

Step	Deliverable
1	Core library (group, Fiat-Shamir, commitments, sharing, base OT, GC) with tests
2	v0 and attacks 1, 2, 4
3	Module D (sigma toolkit, credentials, range proofs) and attacks 7-11, 13
4	Module A (DH-PSI) and attacks 3, 12
5	Module B (Beaver, GC comparison) and attack 6
6	Module C1 (linear and square-root ORAM) with attack 5, then C2 (read-only DORAM)
7	MPC-in-the-head, benchmarks, simulator demos, report, paper-to-module table, slides

Minimum viable core: core library, Module D, Module A, v0, and the attack lab. Modules B and C and MPC-in-the-head are the next layers.

All optional and stretch items in one place

Area	Optional item	Effort	Value for the grade
Core	IKNP OT extension	Medium	High: directly in syllabus
D	1-private MPCitH first, then 3-party	Medium	High
D	Frozen Heart reproduction	Medium	High (research depth)
D	Blind issuance / BBS+ (design only)	Low	Medium
A	OT-based PSI and circuit-PSI	Medium	Medium
B	3-party replicated version	Medium	High
B	GMW comparator with 4-out-of-1 OT	Medium	Medium
B	Explicit Y2B, B2Y, B2A conversions	Low	Medium
B	Shamir shares with degree reduction	Medium	Medium
C	Tree ORAM with recursive position map	High	High
C	Hierarchical ORAM	High	Low (theory-only is fine)
C	DPF and full DORAM	High	High
Attacks	Lattice attack on biased nonces, IKNP correlation, malicious garbler	Medium	High
Demo	Small web dashboard	Low	Medium

14. Research foundation (verify before citing)

These are named from memory. Confirm exact titles, authors and years before they appear in the report. Each entry should appear in the paper-to-module table with the deviations you made and why.

Module	Papers and sources
Course base	Course slides (Vadapalli, CS670): MPC, Secure Memory Access, ZKP
A: PSI	Meadows' early work on PSI; Pinkas, Schneider, Zohner on OT-based PSI; Huberman et al. on DH-style PSI
B: MPC	Yao (garbled circuits); Beaver (multiplication triples); Du and Atallah (secure dot product); free-XOR and GRR3 garbling papers; Goldreich-Micali-Wigderson (GMW); Ishai-Kilian-Nissim-Petrank and IKNP (OT extension)
C: ORAM	Goldreich and Ostrovsky (square-root ORAM); Stefanov et al. (Path ORAM); Shi et al. (tree-based ORAM); Doerner and shelat and related DORAM work; Boyle-Gilboa-Ishai (distributed point functions)
D: ZKP	Schnorr (identification and signatures); Fiat and Shamir; Cramer-Damgard-Schoenmakers (OR-proofs); Chaum-Pedersen (DLEQ); Pedersen (commitments); Ishai-Kushilevitz-Ostrovsky-Sahai (MPC-in-the-head)
Attacks	Dao, Miller, Wright, Grubbs ('Frozen Heart'); Howgrave-Graham and Smart (lattice attacks on partially known nonces); Bernhard, Pereira, Warinschi (weak Fiat-Shamir)
Context	zkKYC (Pauwels et al.); Chaum on blind signatures and 'security without identification'; Aadhaar offline e-KYC and DigiLocker as an Indian case study

Module	Papers and sources
Future work	Camenisch-Lysyanskaya and BBS+ credentials; Coconut; Bulletproofs; Groth16 (all out of syllabus)

15. Open decisions

- **Deadline and team size** (decides where the cut line falls and how many optional items are realistic).
- **Instructor rules** on what must be written from scratch versus library calls.
- **Language preference:** Python is assumed; Rust is possible if performance matters more than speed of iteration.
- **Demo format:** CLI side-by-side is the default; a small web dashboard is optional.